ANÁLISE COMPARATIVA DO USO DO REDIS COMO CAMADA DE CACHE EM UMA ARQUITETURA SAAS MULTI-INQUILINO

Rhyan Willemann Orben[1]

Eduardo Cizeski Meneghel[2]

Resumo: o resumo é uma apresentação concisa de pontos relevantes de um texto na forma de uma sequência corrente de frases (RAUEN, 2015). Deve ser informativo e apresentar objetivos, metodologia, resultados e conclusões. Deve ser escrito em parágrafo único, em espaçamento simples e justificado.

Palavras-chave: Artigo científico. Resumo. Metodologia científica.

1 INTRODUÇÃO

O avanço da computação em nuvem modificou significativamente a forma como aplicações são desenvolvidas, disponibilizadas e consumidas, favorecendo a consolidação do modelo *Software as a Service* (SaaS). Nesse modelo, aplicações são disponibilizadas por meio da infraestrutura do provedor e acessadas sob demanda por diferentes clientes. Entre as características associadas ao SaaS, destaca-se a multi-inquilinidade (*multi-tenancy*), que permite o compartilhamento de recursos de hardware e software entre diferentes organizações, contribuindo para a redução de custos de operação e manutenção (HATTAB; BELALEM, 2023).

Uma das formas de implementação da arquitetura multi-inquilino é o modelo *Single Instance Multi-Tenant* (SIMT), no qual diferentes inquilinos utilizam simultaneamente uma mesma instância da aplicação. Embora esse compartilhamento proporcione benefícios relacionados ao aproveitamento de recursos e à escalabilidade da solução, ele também exige mecanismos que garantam a separação adequada das informações pertencentes a cada cliente. Batista et al. (2024) destacam que o isolamento dos dados constitui um requisito fundamental em sistemas multi-inquilino, especialmente quando informações de diferentes organizações são processadas por uma infraestrutura compartilhada. De maneira complementar, Maciel et al. (2025) apontam que ambientes multi-inquilinos apresentam desafios relacionados à alocação

1 Graduando em Engenharia de Computação no semestre letivo de 2026-02. E-mail: rhyanorben@gmail.com

2 Prof. do Centro Universitário UniSATC. E-mail: eduardo.cizeski@satc.edu.br

de recursos, como CPU e memória, bem como à garantia de isolamento, eficiência e escalabilidade entre cargas concorrentes.

O compartilhamento também pode ocorrer na camada de persistência. Hui et al. (2009) descrevem uma estratégia denominada *Shared Tables and Shared Database Instances* (STSI), na qual diferentes inquilinos armazenam seus registros nas mesmas tabelas de uma única instância de banco de dados. Para permitir a identificação do proprietário de cada registro, os dados são associados a um identificador do inquilino, como uma coluna tenant_id, que também deve ser considerada durante a execução das consultas. Apesar de proporcionar elevado grau de consolidação, o crescimento do volume de dados compartilhados introduz desafios relacionados à eficiência das consultas, à indexação e à escalabilidade do sistema (HUI et al., 2009).

Além do volume de dados, aplicações multi-inquilino estão sujeitas à concorrência pelos recursos computacionais disponíveis. Narasayya et al. (2013) apontam que cargas de trabalho pertencentes a diferentes inquilinos podem disputar recursos de um mesmo servidor de banco de dados, como CPU, memória e operações de entrada e saída, fazendo com que o desempenho de um cliente seja influenciado pelas cargas executadas simultaneamente pelos demais. Em ambientes compartilhados, Maciel et al. (2025) também ressaltam a importância do gerenciamento de CPU e memória para evitar sobrecarga e promover o balanceamento entre as cargas dos diferentes inquilinos. Esse cenário torna-se particularmente relevante à medida que aumenta o volume de dados processados. Zulfa et al. (2020) observam que o crescimento dos dados relacionais e a utilização de consultas envolvendo múltiplas tabelas podem ampliar o tempo necessário para recuperação das informações.

Uma alternativa para reduzir a frequência de acesso ao banco de dados principal consiste na utilização de mecanismos de cache em memória. Segundo Lazoti (2014), o Redis pode ser utilizado como um repositório de cache para dados acessados com frequência e que não sofrem alterações constantes, evitando consultas desnecessárias e contribuindo para a redução do tempo de resposta e do uso de recursos computacionais. De forma complementar, Magalhães, Brayner e Monteiro (2023) destacam que sistemas de bancos de dados em memória são capazes de proporcionar baixa latência e elevadas taxas de vazão justamente por manterem os dados na memória principal. Sistemas *key-value*, como o Redis, podem,

portanto, manter temporariamente dados frequentemente acessados ou resultados cuja obtenção apresenta maior custo, reduzindo a necessidade de consultar repetidamente o armazenamento persistente (PAN et al., 2021).

Resultados experimentais recentes reforçam os benefícios dessa estratégia. Reis, Cardoso e Martins (2023), ao avaliarem a utilização do Amazon ElastiCache para Redis em conjunto com um banco de dados PostgreSQL, identificaram potencial para reduzir consultas redundantes e a latência de acesso, além de aumentar a vazão da aplicação. Mais recentemente, Lavor e Castillo (2026) realizaram testes de carga em uma API com Spring Boot utilizando 120 usuários virtuais e verificaram que a adoção do Redis como cache reduziu em 95% a latência média das operações de leitura e elevou o *throughput* em 113,5% em relação ao cenário sem cache. Embora os experimentos tenham sido realizados em arquiteturas e bancos persistentes distintos, ambos evidenciam o potencial da utilização de uma camada de cache em memória para reduzir o custo de acessos repetitivos aos dados.

Entretanto, a utilização de uma camada de cache acrescenta novos desafios à arquitetura. Em sistemas multi-inquilino, esse gerenciamento torna-se ainda mais relevante, uma vez que diferentes inquilinos podem apresentar volumes de carga, padrões de acesso e características de localidade distintos ao longo do tempo (LIRA, 2023). Quando os dados persistidos no banco principal são alterados, cópias armazenadas em memória podem tornar-se desatualizadas, tornando necessária a aplicação de mecanismos de expiração, atualização ou invalidação. Zheng et al. (2022) demonstram que a manutenção da consistência do cache apresenta custos adicionais e envolve um equilíbrio entre taxa de acertos, carga do servidor e frequência de atualização dos conteúdos armazenados.

Diante desse contexto, estabelece-se a seguinte questão de pesquisa: qual é o impacto da utilização do Redis como camada de cache em memória sobre o tempo de resposta, a vazão e o consumo de recursos computacionais de uma aplicação SaaS baseada em arquitetura SIMT e banco de dados relacional compartilhado, quando submetida a diferentes cargas de leitura e escrita?

Para responder a essa questão, este trabalho tem como objetivo analisar quantitativamente o impacto da utilização do Redis como camada de cache em uma aplicação SaaS multi-inquilino, comparando configurações com e sem cache sob diferentes proporções de operações de leitura e escrita, a partir de métricas de tempo de resposta, vazão e utilização de recursos computacionais.

A realização deste estudo justifica-se pela necessidade de apoiar decisões arquiteturais relacionadas ao desempenho de sistemas SaaS. Embora a adoção de mecanismos de cache possa reduzir a quantidade de acessos ao banco relacional e melhorar o tempo de resposta das aplicações, sua implementação também introduz desafios relacionados ao consumo de memória, à consistência e à invalidação dos dados. Nesse contexto, a avaliação experimental dessas características permite identificar em quais condições a utilização do Redis apresenta benefícios e quais custos são associados à estratégia, contribuindo para a escolha de soluções de cache em aplicações multi-inquilino que utilizam bancos relacionais compartilhados e ampliando a compreensão sobre a relação entre computação em memória, utilização de recursos e desempenho de aplicações.

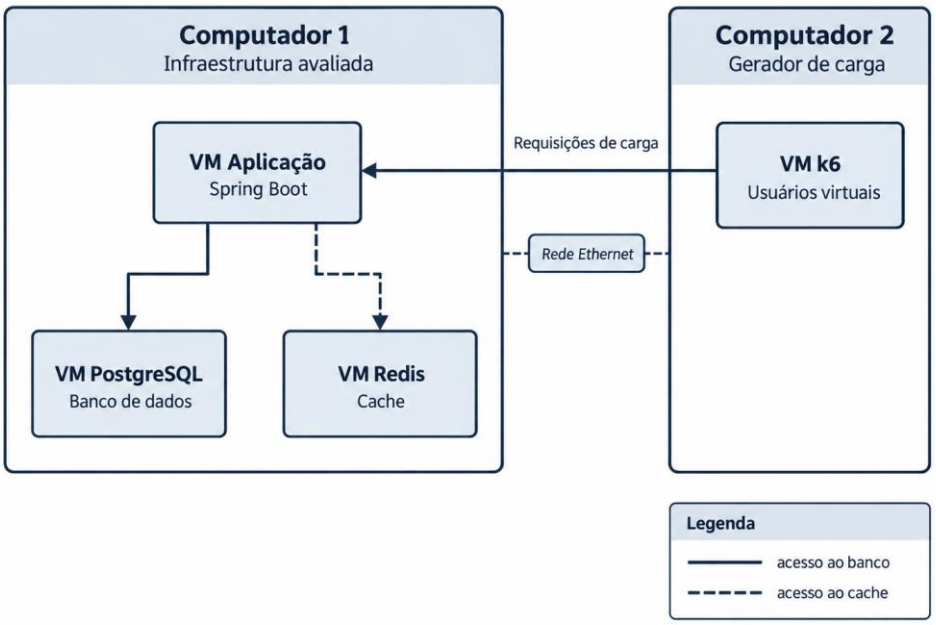
2 PROCEDIMENTO EXPERIMENTAL

Esta seção apresenta os recursos computacionais e os procedimentos adotados para avaliar o impacto da utilização do Redis como camada de cache em uma aplicação SaaS multi-inquilino. O experimento será realizado em ambiente virtualizado e controlado, comparando o comportamento da aplicação em duas configurações: utilizando exclusivamente o PostgreSQL e utilizando PostgreSQL em conjunto com Redis.

2.1 MATERIAIS

Para a realização do experimento serão utilizados dois computadores físicos conectados por rede Ethernet. O primeiro será destinado à execução da infraestrutura avaliada, enquanto o segundo será utilizado exclusivamente para geração das cargas de trabalho. A organização do ambiente experimental é apresentada na Figura 1.

Figura 1: Arquitetura do ambiente experimental



Fonte: Elaborado pelo autor com auxílio de IA generativa (OpenAI, 2026).

Quadro 1: Configurações das máquinas virtuais

Função	vCpu	RAM	Tecnologia e versões
Aplicação	2	4 GB	Java 21; Spring Boot 3.5.16
Banco de Dados	2	4 GB	PostgreSQL 17.11
Cache	1	2 GB	Redis 7.4.11
Geração de Carga	4	4 GB	K6 1.6.0

Fonte: O Autor (2026)

Nota: Ubuntu Server 24.04 LTS em todas as VMs; Docker Engine 29.9.1 e Docker Compose 5.6.0 nas VMs dos serviços.

O primeiro computador físico, responsável pela infraestrutura avaliada, será equipado com processador Intel Core i7-13620H, 32 GB de memória RAM e SSD de 512 GB. O segundo computador, destinado à geração de carga, utilizará processador Intel Core i5-9400F, 16 GB de memória RAM e SSD de 1 TB.

As quatro máquinas virtuais serão criadas por meio de KVM/QEMU com libvirt. No primeiro computador serão executadas as VMs destinadas à aplicação Spring Boot, ao PostgreSQL e ao Redis, enquanto o segundo computador hospedará exclusivamente a VM responsável pela geração de carga com o k6. As VMs do primeiro computador possuirão interfaces de rede próprias e serão interligadas por uma rede virtual privada, permitindo a comunicação entre a aplicação, o PostgreSQL e o Redis por meio de TCP/IP.

A utilização de um equipamento físico separado para geração das requisições busca evitar que o processamento necessário à carga de trabalho dispute CPU e memória com a infraestrutura avaliada. A aplicação, o PostgreSQL e o Redis serão executados em contêineres Docker dentro de suas respectivas máquinas virtuais, permitindo manter constantes as versões dos serviços durante o experimento.

O Redis será utilizado exclusivamente como cache, sem gravação dos dados em disco, com limite de memória de 1 GiB e remoção das entradas menos recentemente utilizadas quando esse limite for atingido. As páginas expirarão 60 segundos após a gravação, sem renovação pelas leituras. A confirmação de uma escrita no PostgreSQL invalidará todas as páginas da empresa. Esses parâmetros permanecerão fixos nos testes com cache.

A base experimental utilizará tabelas compartilhadas com as entidades empresa, cliente, pedido e item de pedido. Serão 100 empresas, cada uma com 5.000 clientes, 20.000 pedidos e 60.000 itens, somando 8.500.100 registros na base. Cada pedido terá três itens, com dados e datas determinísticos. As leituras retornarão resumos paginados de pedidos e dados do cliente, sem carregar os itens.

As tabelas utilizarão chaves primárias e estrangeiras, além de índices sobre o atributo tenant_id e sobre os campos empregados nos filtros das consultas avaliadas. A mesma estrutura de índices será mantida em todos os cenários experimentais. A mesma base de dados será utilizada em todas as execuções experimentais, garantindo condições equivalentes para a comparação entre os cenários avaliados.

2.2 MÉTODOS

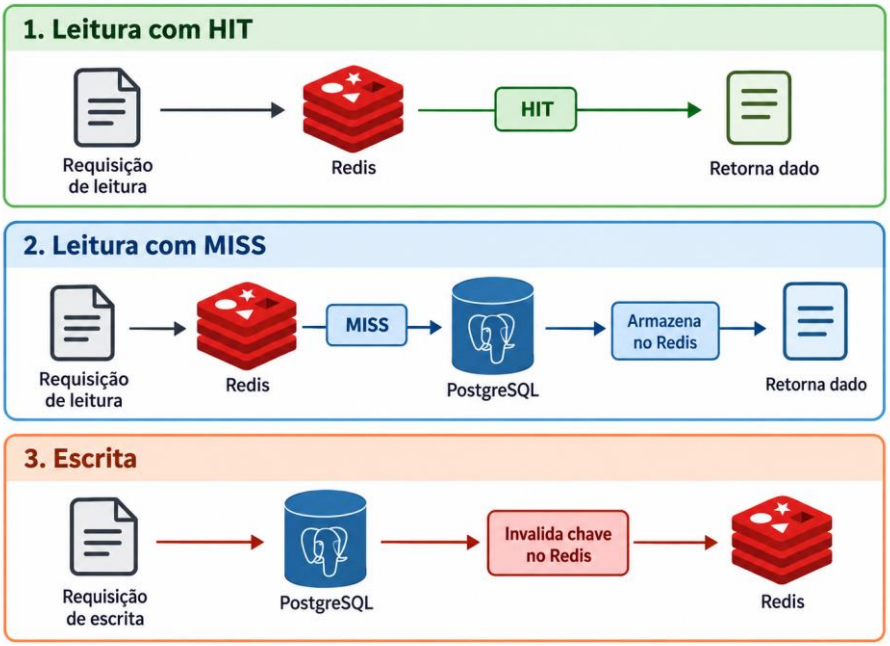
A pesquisa caracteriza-se como aplicada, quantitativa e experimental. O procedimento será baseado na execução controlada da mesma aplicação sob diferentes cargas, comparando os resultados obtidos com e sem a utilização do Redis.

A arquitetura utilizará uma única instância da aplicação Spring Boot compartilhada pelos diferentes inquilinos. Na configuração sem cache, as operações de leitura serão direcionadas diretamente ao PostgreSQL. Na configuração com cache, a aplicação consultará inicialmente o Redis e recorrerá ao PostgreSQL quando a informação não estiver disponível em memória.

Será utilizada a estratégia cache-aside: as páginas de 20 pedidos serão procuradas no Redis e, quando ausentes, consultadas no PostgreSQL e armazenadas

no cache. A ordenação será decrescente por data de criação e identificador. As chaves distinguirão empresa e parâmetros da consulta. Após a confirmação da escrita, será alterada a versão do cache da empresa, invalidando todas as suas páginas; as antigas ficarão inacessíveis até sua remoção ou expiração. Falhas do cache direcionarão a leitura ao banco; entre a escrita e a invalidação, podem ocorrer leituras desatualizadas (Zheng et al., 2022). A Figura 2 apresenta esse fluxo.

Figura 2: Fluxo de acesso aos dados com utilização do cache



Fonte: Elaborado pelo autor com auxílio de IA generativa (OpenAI, 2026).

Serão avaliados quatro perfis de carga, definidos pelas proporções entre operações de leitura e escrita: 100%/0%, 90%/10%, 70%/30% e 50%/50%. Para cada perfil serão utilizados cinco níveis de concorrência, conforme apresentado na Tabela 1.

Tabela 1: Cenários experimentais

Perfil	Leituras	Escritas	Usuários Virtuais
P1	100%	0%	25, 50, 100, 250 e 500
P2	90%	10%	25, 50, 100, 250 e 500
P3	70%	30%	25, 50, 100, 250 e 500
P4	50%	50%	25, 50, 100, 250 e 500

Fonte: Do autor (2026)

A utilização de diferentes volumes de carga permite observar a evolução do comportamento da aplicação à medida que aumenta a concorrência, abordagem também encontrada em avaliações experimentais recentes de serviços e APIs (NISWAR et al., 2024).

Os usuários virtuais serão distribuídos igualmente entre as mesmas 25 empresas em todos os níveis de concorrência; as demais permanecerão apenas na base. Cerca de 80% das leituras consultarão um conjunto fixo de 20% das páginas de cada empresa ativa, e as restantes acessarão as outras páginas. As escritas alternarão o estado de um pedido exclusivo de cada usuário, produzindo alterações reais sem inserir ou excluir pedidos. Empresas, páginas, pedidos de escrita e parâmetros serão preservados entre os modos comparados.

Cada combinação será executada com e sem cache, em cinco repetições, totalizando 200 execuções pareadas, organizadas em 100 pares. Cada par usará a mesma base, parâmetros e semente de seleção, com a ordem dos modos aleatorizada. A duração fixa permite quantidades diferentes de requisições entre os modos.

Antes de cada execução, será aguardada a conclusão das requisições anteriores. O banco experimental será restaurado da mesma cópia de referência, e o Redis dedicado será esvaziado, inclusive antes dos testes sem cache. Nenhuma outra aplicação acessará esses serviços durante a campanha.

Cada execução terá dois minutos de aquecimento somente com leituras, excluídos da análise, seguidos de cinco minutos de medição com o perfil correspondente. Após cada etapa, será aguardada a conclusão das requisições em andamento. O cache começará vazio antes do aquecimento e manterá a expiração de 60 segundos durante ambas as etapas.

As métricas e os instrumentos constam no Quadro 2, com as medidas das requisições separadas por leitura e escrita. A coleta excluirá o aquecimento, utilizará relógios sincronizados e amostrará recursos a cada cinco segundos. Os contadores serão comparados antes da medição e após a conclusão das requisições. A taxa de acerto será a razão entre acertos e a soma de acertos, faltas e desvios ao banco por falha do cache. Estatísticas internas do Redis serão auxiliares. Estudos de desempenho utilizam tempo de resposta e CPU para comparar configurações sob carga (Niswar et al., 2024).

As cinco repetições válidas serão consolidadas por perfil, concorrência, modo e operação, com média, mediana, desvio-padrão e variação percentual entre os modos de cada par. Os percentis serão resumidos por execução, sem tratá-los como percentis de todas as requisições reunidas. Serão apresentados gráficos de concorrência e métricas. Execuções incompletas serão registradas e repetidas. Veja as métricas e instrumentos de coleta no Quadro 2 a seguir.

Quadro 2: Métricas e instrumentos de coleta

Grupo	**Métrica**	**Coleta**
Requisições	Tempo de resposta	k6
Requisições	Percentis p95 e p99	k6
Requisições	Vazão	k6
Requisições	Taxa de erros	k6
Aplicação	CPU e memória	Docker
PostgreSQL	CPU e memória	Docker
Redis	CPU e memória	Docker
PostgreSQL	Comandos SQL da aplicação	Estatísticas do Banco
Cache	Acertos e faltas	Aplicação
Cache	Desvios ao banco por falha	Aplicação
Cache	Memória utilizada	Redis
Cache	Remoções por falta de memória	Redis

Fonte: O Autor (2026)

3 RESULTADOS E DISCUSSÕES

Nos Resultados e Discussões, deve-se apresentar os resultados obtidos no Procedimento Experimental e fazer uma discussão e análise sobre os mesmos sempre que possível referenciando a literatura pesquisada.

4 CONSIDERAÇÕES FINAIS

Etapa esta que servirá para você evidenciar as conquistas alcançadas com o estudo e indicar as limitações e as reconsiderações. Além disso, você poderá apontar a relação entre fatos verificados e teoria e mostrar a contribuição da pesquisa para o meio acadêmico, empresarial e/ou para o desenvolvimento da ciência e tecnologia. Além disso, você poderá sugerir temas complementares a sua pesquisa para estudos futuros. Responda aqui a sua pergunta-problema de pesquisa.

UNI SATC CENTRO UNIVERSITÁRIO

REFERÊNCIAS

BATISTA, Cesar; MORAIS, Felipe; CAVALCANTE, Everton; BATISTA, Thais; PROENÇA, Bruno; CAVALCANTE, William Breno Rodrigues. Managing asynchronous workloads in a multi-tenant microservice enterprise environment. **Software: Practice and Experience**, v. 54, n. 2, p. 334-359, 2024. DOI: 10.1002/spe.3278.

HATTAB, Noureddine; BELALEM, Ghalem. Modular models for systems based on multi-tenant services: a multi-level Petri-net-based approach. **Journal of King Saud University - Computer and Information Sciences**, v. 35, art. 101671, 2023. DOI: 10.1016/j.jksuci.2023.101671.

HUI, Mei; JIANG, Dawei; LI, Guoliang; ZHOU, Yuan. Supporting database applications as a service. In: IEEE INTERNATIONAL CONFERENCE ON DATA ENGINEERING, 25., 2009. **Proceedings [...]**. [S. l.]: IEEE, 2009. p. 832-843. DOI: 10.1109/ICDE.2009.82.

LAVOR, Rubens Heryson de Lima; CASTILLO, Romulo. Análise de performance e segurança em APIs RESTful: um estudo de caso sobre autenticação stateless com JWT e estratégias de cache com Redis. In: CONGRESSO BRASILEIRO INTERDISCIPLINAR EM CIÊNCIA E TECNOLOGIA, 7., 2026. **Anais [...]**. 2026. Disponível em: https://www.even3.com.br/anais/cobicet2026-628540/1653277-analise-de-performance-e-seguranca-em-apis-restful--um-estudo-de-caso-sobre-autenticacao-stateless-com-jwt-e-est/. Acesso em: 9 set. 2026.

LAZOTI, Rodrigo. **Armazenando dados com Redis**. São Paulo: Casa do Código, 2014.

LIRA, Anna Beatriz Lucena. **Sem conflito no cache: observações de uma plataforma de comércio eletrônico multi-tenant**. 2023. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação) — Universidade Federal de Campina Grande, Campina Grande, 2023. Disponível em: https://dspace.sti.ufcg.edu.br/handle/riufcg/36708. Acesso em: 9 set. 2026.

MACIEL, Gabriel E. Bessa; OLIVEIRA JUNIOR, Antonio; GOMES, Raphael; FREITAS, Leandro; SANTOS, Carlos Eduardo Silva. Estratégia de alocação dinâmica de recursos no Kubernetes para ambientes multi-inquilinos. In: SIMPÓSIO BRASILEIRO DE TELECOMUNICAÇÕES E PROCESSAMENTO DE SINAIS, 43., 2025. **Anais [...]**. 2025. Disponível em: https://www.scilit.com/publications/4dcb3fe60c6077ff33300157075c79ff. Acesso em: 9 set. 2026.

MAGALHÃES, Arlino; BRAYNER, Ângelo; MONTEIRO, José Maria. Bancos de dados em memória e suas estratégias de recuperação após falhas. In: JORNADA DE ATUALIZAÇÃO EM INFORMÁTICA 2023. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2023. p. 159-208. https://csbc.sbc.org.br/2023/wp-content/uploads/2023/08/JAI-4-Bancos-de-Dados-em-Memoria-e-suas-Estrategias-de-Recuperacao-Apos-Falhas.pdf. Acesso em: 9 set. 2026.

NARASAYYA, Vivek; DAS, Sudipto; SYAMALA, Manoj; CHANDRAMOULI, Badrish; CHAUDHURI, Surajit. SQLVM: performance isolation in multi-tenant relational Database-as-a-Service. In: BIENNIAL CONFERENCE ON INNOVATIVE DATA SYSTEMS RESEARCH, 6., 2013. **Proceedings [...]**. 2013. Disponível em: https://www.microsoft.com/en-us/research/publication/sqlvm-performance-isolation-in-multi-tenant-relational-database-as-a-service/. Acesso em: 6 set. 2026.

NISWAR, Muhammad; SAFRUDDIN, Reza Arisandy; BUSTAMIN, Anugrayani; ASWAD, Iqra. Performance evaluation of microservices communication with REST, GraphQL, and gRPC. **International Journal of Electronics and Telecommunications**, v. 70, n. 2, p. 429-436, 2024. DOI: 10.24425/ijet.2024.149562.

PAN, Cheng; WANG, Xiaomi; LUO, Yingwei; WANG, Zhenlin. Penalty- and locality-aware memory allocation in Redis using enhanced AET. **ACM Transactions on Storage**, v. 17, n. 2, art. 15, 2021. DOI: 10.1145/3447573.

REIS, Eduardo Guermandi; CARDOSO, Rachel Rodrigues; MARTINS, Sthefany Gabrielli de Siqueira. **Análise de desempenho de um banco de dados PostgreSQL utilizando ElastiCache para Redis**. 2023. Trabalho de Conclusão de Curso (Curso Superior de Tecnologia em Banco de Dados) — Faculdade de Tecnologia de Bauru, Bauru, 2023. Disponível em: https://ric.cps.sp.gov.br/handle/123456789/15428. Acesso em: 9 set. 2026.

ZHENG, Quan; YANG, Tao; KAN, Yuanzhi; TAN, Xiaobin; YANG, Jian; JIANG, Xiaofeng. On the analysis of cache invalidation with LRU replacement. **IEEE Transactions on Parallel and Distributed Systems**, v. 33, n. 3, p. 654-666, 2022. DOI: 10.1109/TPDS.2021.3098459.

ZULFA, Mulki Indana; FADLI, Ari; WARDHANA, Arief Wisnu. Application caching strategy based on in-memory using Redis server to accelerate relational data access. **Jurnal Teknologi dan Sistem Komputer**, v. 8, n. 2, p. 157-163, 2020. DOI: 10.14710/jtsiskom.8.2.2020.157-163.